

To połączenie tworzy **zamknięty obieg danych (Closed-Loop Data System)**. NexusAI staje się "hurtownią dowodów", a Roboton_reflekton "maszyną wykonawczą". W takim układzie księgowy jako człowiek jest potrzebny już tylko do zatwierdzenia finalnego raportu (tzw. *final check*).

Oto jak zaprojektować ten most komunikacyjny, aby dane płynęły bez błędów pod koniec każdego miesiąca:

1. Most Komunikacyjny: Protokół "EOM" (End-Of-Month)

Zamiast ręcznego przesyłania plików, użyjemy mechanizmu **Scheduled Task** (Zadania Harmonogramowane) w Twoim worker.py.

- **NexusAI (Data Producer):** Pod koniec miesiąca agreguje dane z baz SQLite i DuckDB.
- **NATS (Carrier):** Przesyła "Paczka Księgowa" (Accounting Pack) do Roboton_reflekton.
- **Roboton_reflekton (Consumer):** Odbiera paczkę, nakłada na nią filtry z modułu Company_creator i "wypluwa" gotowe dekrety.

2. Agregacja w NexusAI (Przykład zapytania DuckDB)

NexusAI musi przygotować dane w formacie, który Roboton zrozumie. Wykorzystujemy do tego Twoją bazę analityczną **DuckDB**, która w ułamku sekundy podsumuje cały miesiąc.

Python

```
# Funkcja w NexusAI przygotowująca dane dla Robotona
def prepare_monthly_summary(month: int, year: int):
    import duckdb
    con = duckdb.connect('nexus_olap.duckdb')

    # Agregujemy faktury według stawek VAT i kategorii
    summary = con.execute(f"""
    SELECT
        contractor_nip,
        category,
        sum(amount_net) as total_net,
        sum(amount_gross) as total_gross,
        vat_rate
    FROM invoices
    WHERE month(issue_date) = {month} AND year(issue_date) = {year}
    GROUP BY contractor_nip, category, vat_rate
    """).df()
```

```
return summary.to_dict(orient='records')
```

3. Logika Roboton_reflekton: Silnik Dekretacji

Gdy Roboton otrzyma te dane, uruchamia swój moduł logiczny. To tutaj dzieje się "magia", która zastępuje 90% pracy księgowego.

Krok A: Dopasowanie do profilu firmy (Company_creator)

Roboton sprawdza: "Czy ta firma jest na Ryczałcie?". Jeśli tak, ignoruje koszty (netto/brutto) i liczy tylko przychód. Jeśli to Spółka Akcyjna – uruchamia pełną księgowość (Wn/Ma).

Krok B: Automatyczne księgowanie

Python

```
class RobotonBookkeeper:
    def process_nexus_data(self, nexus_data, company_config):
        for record in nexus_data:
            # Roboton decyduje, gdzie wpisać kwotę
            account_wn = self.determine_account_wn(record['category'], company_config)
            account_ma = "201" # Rozrachunki z dostawcami (standard)

            # Zapis do wirtualnej księgi
            self.save_to_ledger(
                date=datetime.now(),
                description=f"Faktura od {record['contractor_nip']}",
                amount=record['total_gross'],
                wn=account_wn,
                ma=account_ma
            )
```

4. Co można w tym systemie zrobić? (Twoje nowe możliwości)

1. **Audyt w 5 sekund:** Zamiast czekać na telefon od księgowej, w dowolnym dniu miesiąca klikasz "Symulacja Roboton" i widzisz, ile podatku masz do zapłacenia *na ten moment*.
2. **Automatyczny JPK_V7:** Roboton pod koniec miesiąca generuje gotowy plik XML, który NexusAI może przechowywać w swojej bazie jako "dokument wyjściowy".

3. **Wykrywanie Optymalizacji:** Roboton może odpowiedzieć: *"Widzę, że w tym miesiącu przekroczyłeś limit X, sugeruję przejście na podatek liniowy od przyszłego roku"* – to jest funkcja doradcy, za którą firmy płacą tysiące złotych.

Jak to połączyć technicznie w PyCharm?

1. **NexusAI** zostaje w Twoim obecnym folderze.
2. Tworzysz nowy folder obok: 📁 Roboton_reflekton.
3. W pliku config.py obu aplikacji ustawiasz **ten sam adres serwera NATS** (localhost:4222).
4. W NexusAI dodajesz skrypt publish_to_roboton.py, który wysyła dane, a w Robotonie subscribe_to_nexus.py, który na nie czeka.

Czy chcesz, żebym przygotował specyfikację bazy danych dla Roboton_reflekton, która będzie przechowywać te "Księgi Rachunkowe" w formacie SQL?